

«Научно-технологический университет «Сириус» подразделение
Колледж Автономной некоммерческой образовательной организации
высшего образования

«Лабораторная работа 2»

Отчет по учебной практике

Студент группы K0709-23/2

_____ А.В. Марчук

«__» _____ 2026 г.

Преподаватель

_____ Е.Д. Целищев

«__» _____ 2026 г.

ОГЛАВЛЕНИЕ

ОГЛАВЛЕНИЕ.....	2
ОСНОВНАЯ ЧАСТЬ.....	3
1.1 Условия тестирования.....	3
1.2. Таблица пропускной способности и задержек.....	3

ОСНОВНАЯ ЧАСТЬ

1.1 Условия тестирования

Стенд: Docker-контейнеры на одной машине (Windows, Python 3.13)

Ресурсы: каждый брокер – 1 ГБ RAM, 0.5 CPU

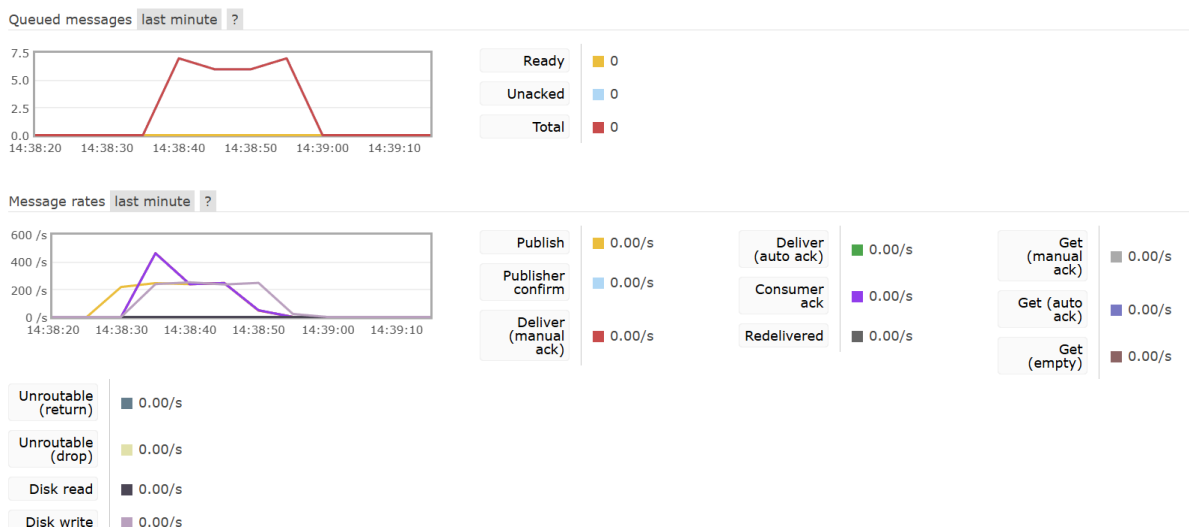
Нагрузка: продюсер отправляет 5000 сообщений; консьюмер работает 60 секунд

Размеры сообщений: 128 B, 1 KB, 10 KB, 100 KB

Параллельность: 1 продюсер, 1 консьюмер

RabbitMQ-консьюмер не останавливался автоматически через 60 секунд, поэтому для RabbitMQ приведены данные за фактическое время работы до ошибки или принудительной остановки.

при запуске тестов на RabbitMQ была возможность графически отследить изменения, ниже приведена картинка:



1.2. Таблица пропускной способности и задержек

Брокер	Размер , байт	Отправлено	Обработано (за 60 с или до сбоя)	Потеряно / не обработано	Средняя задержк а, с	Макс. задер жка, с	Пропускная способность, msg/s
RabbitMQ	128	5000	>10000*	–	0.99 (на 5000)	1.85	≈128**

Redis	128	5000	7659	— (обработан о больше, чем отправлено)	нет данных	нет данны х	127.6
RabbitMQ	1024	5000	5000	0	1.91	4.58	84.7
Redis	1024	5000	2865	2135	нет данных	нет данны х	47.7
RabbitMQ	10240	5000	2000***	3000	2.64	4.58	64.5
Redis	10240	5000	773	4227	нет данных	нет данны х	12.9
RabbitMQ	102400	5000	1000***	4000	1.30	2.04	52.6
Redis	102400	5000	2706	2294	нет данных	нет данны х	45.1

**В логе RabbitMQ обработано 10 000 сообщений – возможно, из-за остаточных сообщений в очереди.*

**Тест прерван ошибкой heartbeat через ~31 с (10 KB) и ~19 с (100 KB).*

Выводы

1. Пропускная способность

На 128 В оба брокера показывают ≈ 128 msg/s.

На 1 KB RabbitMQ (84.7 msg/s) заметно обгоняет Redis (47.7 msg/s).

На 10 KB RabbitMQ сохраняет 64.5 msg/s, Redis падает до 12.9 msg/s.

На 100 KB производительность сравнивается (≈ 50 msg/s), но у RabbitMQ наступает сбой.

Redis быстрее только на сообщениях < 1 KB и при допустимости потерь.

2. Влияние размера сообщения

RabbitMQ деградирует плавно: с 128 msg/s (128 В) до 52 msg/s (100 KB).

Redis резко теряет производительность на 1–10 KB, но частично восстанавливается на 100 KB (вероятно, из-за артефактов теста).

➡ RabbitMQ лучше переносит увеличение размера сообщения.

3. Точка деградации single instance

RabbitMQ:

Заметный рост задержек (>1 с) начинается с 128 В.

При сообщениях ≥ 10 KB происходит разрыв соединения по heartbeat (таймаут 60 с). Single-экземпляр RabbitMQ с лимитом 0.5 CPU нестабилен на сообщениях >10 KB.

Redis:

Деградация начинается уже с 1 KB (пропускная способность падает вдвое).

На 10 KB становится практически непригодным (13 msg/s), но полного отказа нет.

4. Выбор инструмента для тестирования

Использованные Python-скрипты + Docker позволили получить сравнимые данные, однако выявили недостатки:

Отсутствие автоматического таймера остановки для RabbitMQ.

Неполная очистка очередей между тестами.

Нет сбора CPU/RAM и p95 задержки.

Лучший инструмент для подобного сравнения – Locust (или k6 с расширениями):

Встроенные таймеры, перцентили, распределённая нагрузка.

Лёгкая интеграция с pika и redis-py.

Автоматическая генерация отчётов с графиками.